

CSCN8020 Full Presentation Transcript: Slides 1 to 10

Course: CSCN8020 - Reinforcement Learning | **Project:** Namaste-G1 Humanoid Gesture Learning
Group 4: Ali Cihan Ozdemir, Lohith Reddy Danda, Sumanth Reddy Konannagari, Muthuraj Jayakumar
Total Slides: 10 | **Target Duration per Slide:** 45 Seconds (~95 words)

■ **Recommended Presenter Division (4 Group Members):**

Presenter	Slides	Topic / Focus	Time
Presenter 1	Slides 1 & 2	Introduction, Project Overview & Roadmap	1 min 30s (2x45s)
Presenter 2	Slides 3 & 4	DQN Baseline Results & Transition to Actor-Critic	1 min 30s (2x45s)
Presenter 3 (Ali)	Slides 5, 6 & 7	Headline Finding, Zero-Gradient Bug & Post-Fix Results	2 min 15s (3x45s)
Presenter 4	Slides 8, 9 & 10	Compute Metrics, Mathematical Comparison & Live Demo	2 min 15s (3x45s)

■ ■ **SLIDE 1: Title & Project Overview**

■ ■ **Target:** 45s | ■ **Word Count:** 95 words | ■ ■ **Action:** Point to title & team names

Spoken Transcript:
Good morning Professor Espinosa and class. I'm Ali Cihan Ozdemir, and together with my team members Lohith, Sumanth, and Muthuraj, we are Group 4.

Today, we're taking reinforcement learning beyond basic joint movement to solve a complex robotics challenge:
Namaste-G1: Teaching a Humanoid Robot to Greet.

Our goal is to train the 14-degree-of-freedom Unitree G1 humanoid robot to execute a full two-arm Namaste greeting—reaching the pose, holding it steadily within strict tolerances, and returning gracefully to rest.

Here is how we tackled multi-joint humanoid control here at Conestoga College.

■ ■ **SLIDE 2: Today’s Plan (Agenda)**

■ ■ **Target:** 45s | ■ **Word Count:** 95 words | ■ ■ **Action:** Point to roadmap items

Spoken Transcript:
Here is the roadmap for our presentation today.

We will begin by reviewing our initial 14-joint DQN baseline, and explain why we transitioned from discrete step control to continuous Actor-Critic.

Next, we will reveal our major headline finding and the critical PyTorch gradient bug we discovered along the way.

We will then analyze what happened after fixing the bug—where the robot demonstrated real learning movement but experienced policy instability—followed by explaining the underlying mathematical principles in plain terms, running a live simulation demonstration, and detailing our next steps for the project.

■ ■ **SLIDE 3: Before Actor-Critic: The DQN Baseline**

■ ■ **Target:** 45s | ■ **Word Count:** 95 words | ■ ■ **Action:** Point to 14-joint stats (3.1 to 5.2 joints)

Spoken Transcript:

To set the baseline, we first built a 14-joint branching Dueling Q-Network. Each joint had three discrete actions: hold, decrease, or increase the angle.

We ran a 200-episode diagnostic test, logging every step. While the network learned to improve the average number of joints in tolerance from 3.1 up to 5.2—reaching a peak moment of 7 out of 14 joints—it achieved zero full 14-joint successes.

This proved that coarse, stepped discrete actions made fine multi-joint coordination extremely difficult.

■■ SLIDE 4: Why We Moved to Actor-Critic

■■ Target: 45s | ■ Word Count: 95 words | ■■ Action: Point to DQN Stepped vs Actor-Critic Smooth

Spoken Transcript:

That limitation motivated our shift to Actor-Critic.

While DQN could only move each joint in fixed discrete steps, Actor-Critic outputs a continuous action value for every joint. Finer, smoother control is essential for coordinating 14 joints simultaneously.

However, moving to continuous control made the search space much larger, making the task harder rather than easier.

Our refined goal expanded to three steps: reaching the Namaste pose, holding it for a streak, and returning smoothly to rest.

■■ SLIDE 5: After Actor-Critic: The Headline Finding

■■ Target: 45s | ■ Word Count: 95 words | ■■ Action: Point to headline quote box

Spoken Transcript:

After implementing the complete Actor-Critic pipeline, we ran over 7,800 training episodes cleanly with zero code crashes or NaN values.

However, despite long training runs, our early full experiments still produced zero complete successes.

We soon realized that the main bottleneck was not just slow training convergence. Our critical headline finding was that the Actor network received zero working gradient updates during early runs because a hidden implementation bug completely froze the policy weights.

■■ SLIDE 6: The Bug: `rsample()` + `log_prob()` = Zero Gradient

■■ Target: 45s | ■ Word Count: 95 words | ■■ Action: Point to zero vs flowing gradient flow

Spoken Transcript:

When inspecting network parameters across episodes, we discovered that the Actor weights at episode 50 and episode 300 were byte-identical.

While the Critic network updated normally, the Actor was completely frozen.

The root cause was that calling PyTorch's `rsample()` together with `log_prob()` on the exact same action sample produced a mathematical zero gradient.

Once we verified this zero-gradient behavior through code reproduction, we switched to `dist.sample()` for training actions, which immediately unlocked gradient flow.

■■ SLIDE 7: After the Fix: Real Learning, Still Unstable

■ Target: 45s | ■ Word Count: 95 words | ■ Action: Point to 6 stabilization attempts

Spoken Transcript:

Immediately after fixing the gradient bug, the policy moved for the very first time.

To achieve full pose convergence, we conducted six distinct stabilization experiments, testing lower learning rates, gradient clipping, reward tuning, longer training runs, warm-start initialization, and batching.

Although joint movement improved significantly, the policy oscillated near the target rather than holding a clean pose.

This empirical takeaway proved that vanilla Actor-Critic remains inherently high-variance for 14-joint continuous control.

■■ SLIDE 8: Compute Time & Timeline Breakdown

■ Target: 45s | ■ Word Count: 95 words | ■ Action: Point to 8,250 episodes & 60%+ compute circle

Spoken Transcript:

In total, our team logged 8,250 episodes across 5.4 hours of wall-clock training compute.

Over 60% of our compute budget was spent before finding the gradient bug, running 5,750 episodes while the Actor was mathematically unable to learn.

The primary bottleneck was single-threaded CPU physics simulation in MuJoCo, requiring up to 4,000 steps per episode.

Based on diagnostic signals, we estimate requiring 5,000 to 10,000 additional training episodes to achieve stable convergence.

■■ SLIDE 9: The Math Behind It: DQN vs. Actor-Critic

■ Target: 45s | ■ Word Count: 95 words | ■ Action: Point to DQN TD target vs Actor-Critic Policy Gradient

Spoken Transcript:

Mathematically, the core difference between both algorithms comes down to how actions are learned.

DQN memorizes one scalar value per discrete state-action pair, updating toward a bootstrapped TD target: $y = r + \gamma \max Q(s', a')$.

In contrast, Actor-Critic uses a Critic to estimate state value $V(s)$ and calculates the Advantage $A(s, a) = r + \gamma V(s') - V(s)$ to sharpen continuous policy probabilities.

In short: DQN memorizes exact discrete action values, while Actor-Critic sharpens continuous policies based on outcome surprise.

■■ SLIDE 10: Live Demo Flow

■ Target: 45s | ■ Word Count: 95 words | ■ Action: Point to 3-step demo table

Spoken Transcript:

We will now demonstrate our live simulation pipeline using three specific commands.

First, executing `target_pose.py` displays the ideal target Namaste pose.

Second, running `watch.py` demonstrates our DQN baseline attempting the gesture, reaching up to 7 out of 14 joints simultaneously.

Third, executing `watch_ac.py` showcases our Actor-Critic policy following the gradient bug fix, demonstrating genuine movement toward the pose.

Thank you for your time, and we now welcome any questions!